

CRAFT: Category-aware Reasoning and Adaptive Framework for Vulnerability Types

Sijune Hwang , Sungju Yun , Hong-In Won , Junggab Son , Yeonjoon Lee , Sunbin Park 

Abstract—Software vulnerabilities arise from diverse causal mechanisms, including unsafe program states, erroneous execution flows, constraint violations, and security policy failures. Despite recent advances in Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and multi-agent reasoning, existing vulnerability detection approaches largely employ unified reasoning strategies that overlook these fundamental differences. We argue that effective vulnerability analysis requires reasoning tailored to the underlying causal mechanism of each vulnerability type. To this end, we propose CRAFT (Category-aware Reasoning and Adaptive Framework for Vulnerability Types), a category-aware vulnerability detection framework that organizes vulnerabilities into four causality types: state-dependent, execution-dependent, constraint-dependent, and policy-dependent. CRAFT introduces causality-typed root cause grounding, which represents each vulnerability through a mechanism-specific causal chain and incorporates it into a knowledge graph. Guided by this structured knowledge, a collaborative multi-agent pipeline performs mechanism-aware vulnerability reasoning. Experiments on the PrimeVul benchmark covering representative CWEs from all four causality types demonstrate that CRAFT substantially outperforms existing approaches, improving Pairwise Correct Prediction (P-C) by 51.97% over CLEAR, the strongest baseline. These results show that explicitly modeling causality types enables more accurate and reliable vulnerability reasoning than a unified reasoning paradigm.

Index Terms—Vulnerability Detection, Large Language Models, Causal Reasoning, Knowledge Graph, Software Security.

I. INTRODUCTION

SOFTWARE vulnerabilities remain a persistent threat to modern software systems, whose scale and complexity continue to increase. Identifying security vulnerabilities through manual code inspection is labor-intensive and requires substantial security expertise [1]. Traditional automated approaches have therefore relied extensively on static analysis, but such tools remain prone to false positives and may fail to

detect substantial portions of vulnerabilities across different vulnerability types [2], [3]. More recently, deep learning approaches have learned vulnerability-related representations directly from source code [4], [5], [6]. However, these models can rely on spurious or vulnerability-irrelevant features, limiting their robustness and their ability to distinguish vulnerable functions from their patched counterparts [7], [8], [9].

Large Language Models (LLMs) have recently demonstrated strong capabilities in code understanding and natural language reasoning [10], [11], motivating their application to software vulnerability detection and analysis [12], [13], [14], [15], [16]. Nevertheless, LLM-based vulnerability detection remains unreliable when vulnerable code must be distinguished from its benign or patched counterpart, as security-critical differences are often remarkably subtle [7]. A small modification, such as an added validation check, a corrected condition, or a change to an unsafe operation, can fundamentally alter the security behavior of a program [7], [17]. To improve the reliability of LLM reasoning, recent approaches have incorporated external knowledge through Retrieval-Augmented Generation (RAG) [18], [19], [20], [21], [22] and employed multi-agent collaboration for complementary analysis and verification [23], [24]. However, retrieved evidence is often organized primarily by semantic similarity rather than by the causal relationships underlying vulnerability behavior, while multi-agent collaboration alone cannot ensure reliable reasoning when agents lack appropriately structured security evidence [25], [26], [27].

Our preliminary work, CLEAR (Causal Context-Based Agentic Reasoning for Vulnerability Detection) [28], introduced a Vulnerability Causal Knowledge Graph (VCKG) that represents vulnerability knowledge through a common causal structure comprising execution entrypoints, triggering preconditions, root causes, and fix intentions. By grounding multi-agent reasoning in this structured causal context, CLEAR demonstrated the effectiveness of explicitly incorporating vulnerability causality into LLM-based vulnerability detection. This unified representation provides a general causal abstraction that can be applied across vulnerability categories. However, it does not explicitly distinguish the different causal mechanisms through which those categories arise.

This distinction becomes important because different vulnerability categories require different causal properties to establish their existence. For a NULL Pointer Dereference (CWE-476), reasoning centers on the state of a pointer and the transition that may lead to an invalid dereference. For a Race Condition (CWE-362), the relevant causal properties instead concern interacting execution events and their ordering. Similarly, an Integer Overflow (CWE-189) involves numerical

Y. Lee is the corresponding author.

S. Yun and S. Park are with the Department of Computer Science and Engineering, Hanyang University, Seoul, Republic of Korea (e-mail: {tjdwn77777, pbt98}@hanyang.ac.kr).

S. Hwang is with the Department of Information Security, Hanyang University, Seoul, Republic of Korea (e-mail: ghkdtlwns987@hanyang.ac.kr).

Y. Lee is with the College of Computing, Hanyang University, Ansan, Republic of Korea (e-mail: yeonjoon@hanyang.ac.kr).

H.-I. Won is with the Manufacturing AI Research Center, Korea Institute of Industrial Technology (KITECH), Republic of Korea (e-mail: luvhayym@kitech.re.kr).

J. Son is with the Department of Computer Science, University of Nevada, Las Vegas, Las Vegas, NV 89154, USA (e-mail: junggab.son@unlv.edu).

A preliminary version of this work, entitled "CLEAR: Causal Context-Based Agentic Reasoning for Vulnerability Detection," was accepted for publication in the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE 2026). This article substantially extends the conference version.

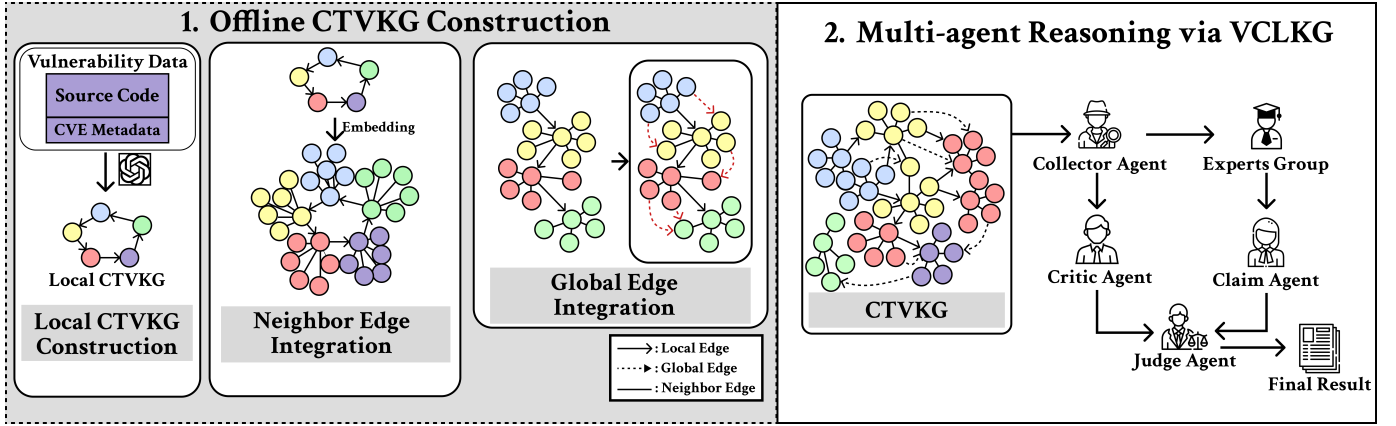


Fig. 1. Overview of the CRAFT framework.

constraints and the operations that may violate them, whereas Improper Access Control (CWE-284) involves protected assets, security policies, and the operations governed by those policies [29]. CLEAR can represent these cases through its common precondition and root-cause abstraction, but their mechanism-specific causal semantics are not explicitly differentiated within that representation. Consequently, the same causal roles may encompass substantially different forms of evidence and reasoning across vulnerability categories.

Building on this observation, we propose **CRAFT** (Category-aware Reasoning and Adaptive Framework for Vulnerability Types), a causality-aware vulnerability detection framework that specializes causal representation, retrieval, and reasoning according to the mechanism underlying each vulnerability. CRAFT organizes vulnerabilities into four causality types: *State*, *Execution*, *Constraint*, and *Policy*. At its core, CRAFT introduces *Causality-Typed Root Cause Grounding*, which specializes a common causal structure according to these mechanisms and represents the resulting knowledge in a Causality-Typed Vulnerability Knowledge Graph (CTVKG). This design allows retrieval to preserve not only the causal role of evidence, but also the mechanism-specific semantics that the evidence represents, such as state transitions, execution flows, constraint violations, and policy violations. CRAFT further employs specialized State, Execution, Constraint, and Policy experts to analyze the retrieved causal contexts according to their corresponding mechanisms. Their hypotheses are subsequently synthesized and adversarially verified by collaborative agents to produce the final vulnerability decision. In this way, CRAFT extends the general causal abstraction established by CLEAR by explicitly incorporating mechanism-specific causal semantics into both evidence grounding and reasoning.

This article substantially extends our preliminary work, CLEAR [28]. Beyond the conference version, CRAFT introduces a causality-aware formulation of vulnerability mechanisms, Causality-Typed Root Cause Grounding, a redesigned CTVKG with causality-specific semantics, and causality-specific expert reasoning. We further provide new architectural ablations and robustness evaluations across multiple LLM

backbones.

In summary, this paper makes the following contributions:

- We propose **CRAFT**, a causality-aware vulnerability detection framework that explicitly distinguishes vulnerability reasoning according to four causality types: *State*, *Execution*, *Constraint*, and *Policy*. CRAFT introduces **Causality-Typed Root Cause Grounding** to encode the distinct causal structure of each causality type within a unified Causality-Typed Vulnerability Knowledge Graph (CTVKG), enabling causality-specific evidence retrieval and reasoning across heterogeneous vulnerability mechanisms.
- We develop **causality-specific expert reasoning**, in which specialized State, Execution, Constraint, and Policy experts analyze retrieved evidence according to its associated causality type. Their mechanism-specific hypotheses are subsequently synthesized and adversarially verified to produce the final vulnerability decision.
- We conduct extensive experiments on the PrimeVul benchmark covering representative CWEs from all four causality types. CRAFT achieves a Pairwise Correct Prediction (P-C) of 22.75%, improving upon the strongest baseline by **51.97%**.

II. METHODOLOGY

In this section, we present the overall architecture of **CRAFT**, a causality-aware vulnerability detection framework that performs multi-agent reasoning using structured causal knowledge represented in a Causality-Typed Vulnerability Knowledge Graph (CTVKG). CRAFT is motivated by the observation that software vulnerabilities arise through heterogeneous causal mechanisms and therefore require different forms of context and reasoning. To capture these differences, CRAFT organizes vulnerability knowledge into four *causality types*: *State*, *Execution*, *Constraint*, and *Policy*, while maintaining a common causal interface across them.

CRAFT consists of two primary processes: **(1) Offline CTVKG Construction** and **(2) Multi-Agent Reasoning via CTVKG**. Fig. 1 illustrates the overall architecture, with the left side showing offline graph construction from historical vulnerability data and the right side showing online causal

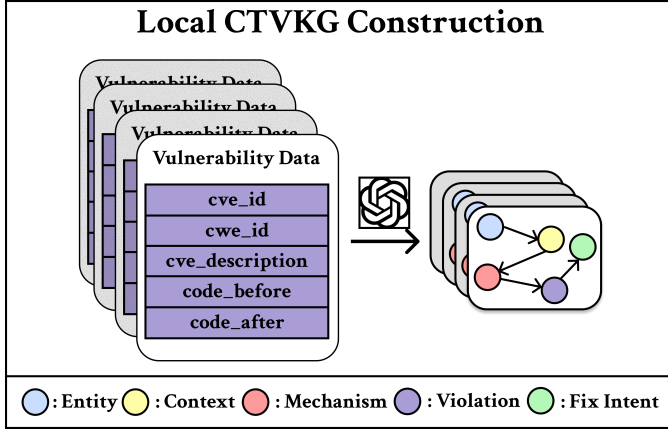


Fig. 2. Local CTVKG construction from a historical vulnerability instance.

reasoning over a target function. The following subsections describe these two processes in detail.

A. Offline CTVKG Construction

As shown in Fig. 1, CRAFT constructs the CTVKG from historical vulnerability data before online inference. Each historical instance contains vulnerable source code and associated vulnerability information, including its CWE category, CVE description, and patch information.

CTVKG construction proceeds in three stages: **Local CTVKG Construction**, **Neighbor Edge Integration**, and **Global Edge Integration**. Local CTVKG Construction represents the historical vulnerability using a causality-specific node schema. Neighbor Edge Integration connects analogous causal knowledge across related vulnerability instances, and Global Edge Integration introduces cross-CVE causal bridges beyond direct semantic neighborhoods. Together, these stages transform independently represented vulnerability instances into an integrated causality-typed knowledge graph.

1) *Local CTVKG Construction*: Fig. 2 illustrates the construction of a Local CTVKG. Given a historical vulnerability instance with its associated CWE-ID, CRAFT identifies the corresponding causality type according to the taxonomy in Table I. This assignment determines the node schema used to extract causal information from the vulnerability instance.

TABLE I
CAUSALITY-TYPED CWE TAXONOMY

CWE-ID	Description	Causality Type
CWE-476	NULL Pointer Dereference	State
CWE-415	Double Free	State
CWE-401	Memory Leak	State
CWE-835	Infinite Loop	State
CWE-703	Improper Exception Handling	Execution
CWE-617	Reachable Assertion	Execution
CWE-362	Race Condition	Execution
CWE-189	Numeric Errors	Constraint
CWE-369	Divide By Zero	Constraint
CWE-200	Information Exposure	Policy
CWE-284	Improper Access Control	Policy

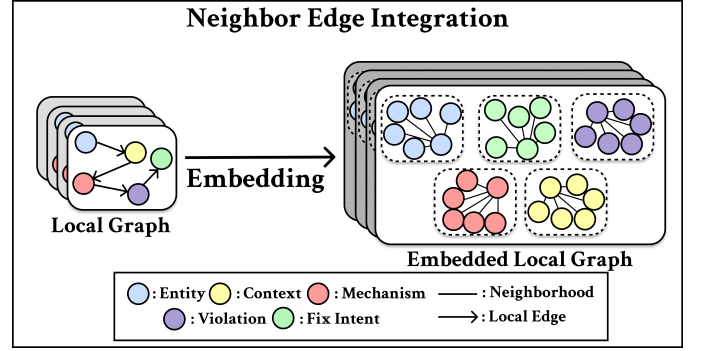


Fig. 3. Neighbor Edge Integration using CWE relationships and type-consistent semantic similarity.

CRAFT applies **Causality-Typed Root Cause Grounding** to represent each vulnerability through five common causal roles:

$$\text{Entity} \rightarrow \text{Context} \rightarrow \text{Mechanism} \rightarrow \text{Violation} \rightarrow \text{Fix Intent}.$$

The *Entity* identifies the primary program element involved in the vulnerability. The *Context* characterizes the condition governing the Entity, such as its program state, execution condition, required constraint, or security policy. The *Mechanism* describes the operation, transition, or execution behavior through which the vulnerability develops. The *Violation* captures the resulting unsafe condition or security relevant failure, while the *Fix Intent* represents the corrective logic introduced by the patch.

These five causal roles provide a common interface across all causality types, while their concrete semantics are specialized for each type, as summarized in Table II.

Using the schema associated with the assigned causality type, the LLM extracts the required causal information from the vulnerable code, patch, CVE description, and vulnerability metadata. For example, for a State-type vulnerability, the LLM extracts *state_entity*, *current_state*, *state_transition*, *invalid_state*, and *fix_intent*. The vulnerable code and CVE description provide evidence for characterizing the vulnerability mechanism, while the patch provides evidence for identifying the corresponding Fix Intent.

The extracted nodes are subsequently normalized and checked for schema completeness to ensure that all required node types are present. They are then connected through **Local Edges** according to the causal structure of the corresponding causality type. The resulting Local CTVKG represents a historical vulnerability as a structured, causality-typed causal chain and serves as the basic unit for the subsequent graph integration stages.

2) *Neighbor Edge Integration*: Local CTVKGs independently represent the causal structure of individual vulnerability instances. CRAFT therefore performs **Neighbor Edge Integration** to connect analogous causal knowledge across related historical vulnerabilities. As illustrated in Fig. 3, this process consists of *CWE-guided candidate expansion* and *semantic neighbor selection*.

For each node, CRAFT constructs a candidate set from nodes associated with the same CWE or related CWE IDs

TABLE II
CAUSALITY-SPECIFIC NODE TYPES FOR THE FIVE COMMON CAUSAL ROLES

Causality Type	Entity	Context	Mechanism	Violation	Fix Intent
State	state_entity	current_state	state_transition	invalid_state	fix_intent
Execution	execution_events	execution_context	execution_flow	unsafe_execution	fix_intent
Constraint	constrained_value	required_constraint	critical_operation	constraint_violation	fix_intent
Policy	protected_asset	security_policy	access_operation	policy_violation	fix_intent

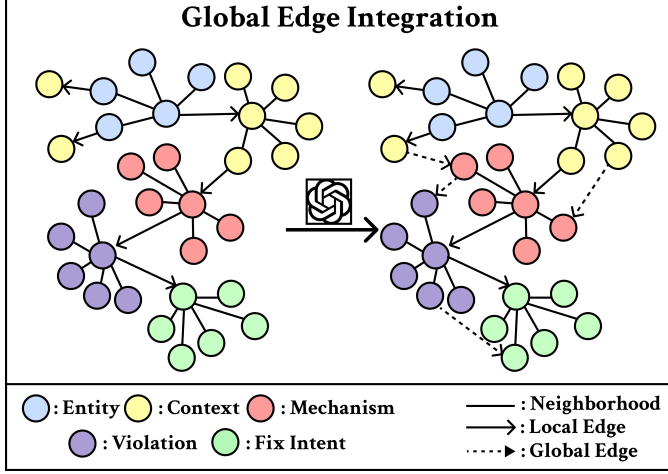


Fig. 4. Global Edge Integration through cross-CVE causal bridge construction.

defined in the MITRE CWE taxonomy [29]. Candidate nodes are further restricted to the same causality-specific node type. For example, a *state_transition* node is compared only with other *state_transition* nodes, preventing comparisons between semantically incompatible node types.

CRAFT then computes semantic embeddings for the candidate nodes and selects the top- k most similar nodes ($k = 5$). The selected node pairs are connected through **Neighbor Edges**, combining CWE-level relationships with node-level semantic similarity to share analogous causal knowledge across Local CTVKGs while preserving causality-type consistency.

3) *Global Edge Integration*: Neighbor Edges establish semantic correspondences between analogous nodes, but they do not capture broader causal connections that may exist across vulnerability instances. CRAFT therefore performs **Global Edge Integration**, illustrated in Fig. 4, to construct causal bridges between Local CTVKGs.

Starting from Local CTVKGs connected through Neighbor Edges, CRAFT identifies candidate cross-instance relations between their causal nodes. Candidate relations are restricted according to both causality type and causality-specific node type to avoid connecting nodes with incompatible causal semantics. An LLM then evaluates the candidate relations in the context of the corresponding vulnerability instances and determines whether they represent valid cross-instance causal connections. Relations validated by the LLM are retained as **Global Edges**.

The resulting CTVKG contains three complementary types of relations: **Local Edges** encode causal dependencies within

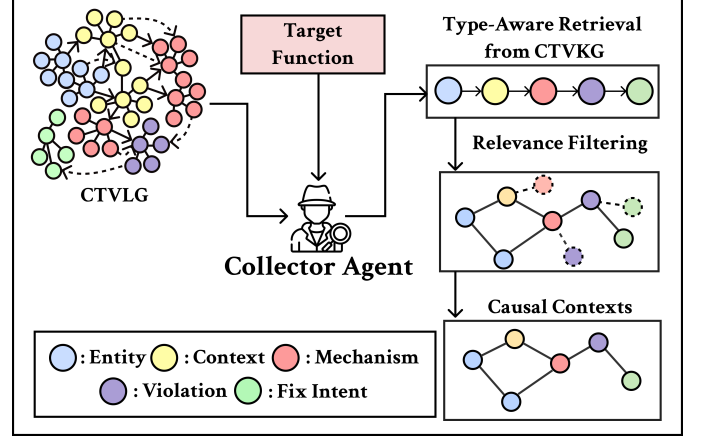


Fig. 5. Causal context retrieval by the Collector Agent.

individual vulnerability instances, **Neighbor Edges** connect semantically analogous causal nodes across related instances, and **Global Edges** establish cross-instance causal bridges beyond direct semantic similarity. Together, these relations form the structured causal knowledge used during online inference.

B. Multi-Agent Reasoning via CTVKG

As shown on the right side of Fig. 1, CRAFT uses the constructed CTVKG as external causal knowledge when analyzing a target function. The online process consists of three conceptual stages: *causal context retrieval*, *causality-specific hypothesis generation*, and *decision making*. The Collector first retrieves causal context from the CTVKG. The Experts Group then generates mechanism-specific hypotheses, which are subsequently synthesized, challenged, and evaluated by the Claim, Critic, and Judge agents.

1) *Collector Agent*: The **Collector Agent** retrieves structured causal context from the CTVKG for a given target function. As illustrated in Fig. 5, retrieval follows the same five causal roles used during graph construction: *Entity*, *Context*, *Mechanism*, *Violation*, and *Fix Intent*. This shared interface enables the Collector to retrieve heterogeneous causality-specific node types through a consistent causal structure.

For each causal role, the Collector generates a dedicated query conditioned on the target code and retrieves candidate nodes from the CTVKG. An LLM then filters these candidates based on their relevance to the target code and consistency with the intended role. The retained nodes preserve their causality-specific semantics and collectively form the causal context for subsequent reasoning.

The retrieved context is separated according to its downstream use. *Entity*, *Context*, *Mechanism*, and *Violation* context is provided to the Experts Group for causality-specific hypothesis generation, whereas *Fix Intent* context is reserved for the Critic Agent as counter-evidence during vulnerability verification.

2) *Experts Group*: The retrieved causal context is analyzed by the **Experts Group**, which consists of four specialized reasoners: the *State Expert*, *Execution Expert*, *Constraint Expert*, and *Policy Expert*. Each expert reasons over the node types associated with the *Entity*, *Context*, *Mechanism*, and *Violation* roles of its corresponding causality type, as defined in Table II.

Expert activation is determined by the causality type associated with the retrieved *Entity* nodes. A *state_entity* activates the State Expert, *execution_events* activates the Execution Expert, *constrained_value* activates the Constraint Expert, and *protected_asset* activates the Policy Expert. Multiple experts may be activated when the retrieved context contains *Entity* nodes from multiple causality types.

Each activated expert analyzes the target function together with the causal context associated with its causality type and generates vulnerability hypotheses. Each hypothesis describes a plausible causal path through which the target function may exhibit the corresponding vulnerability mechanism and retains references to the causal context supporting the hypothesis.

3) *Claim Agent*: The **Claim Agent** receives the target function and the mechanism-specific hypotheses generated by the Experts Group. Its role is to synthesize these hypotheses into candidate vulnerability claims. The Claim Agent consolidates redundant hypotheses, combines complementary causal explanations when appropriate, and preserves references to the supporting expert hypotheses and causal context. This synthesis provides a common representation for subsequent reasoning while allowing hypotheses from multiple causality types to contribute when the target function exhibits more than one plausible

4) *Critic Agent*: The **Critic Agent** performs verification of each candidate claim. Rather than generating an independent vulnerability hypothesis, the Critic examines whether the mechanism proposed by the Claim Agent remains feasible in the target function. In particular, the Critic uses the *Fix Intent* context retrieved by the Collector to identify corrective logic that may interrupt the hypothesized causal path. This distinction is important when analyzing vulnerable and patched functions, which may remain highly similar even after a security critical modification. A validation check, corrected condition, synchronization operation, or other patch behavior may invalidate an otherwise plausible vulnerability hypothesis. When such context contradicts the candidate claim, the Critic produces a corresponding counterargument.

5) *Judge Agent*: Finally, the **Judge Agent** evaluates each candidate claim together with the corresponding counterargument. The Judge determines whether the retrieved causal context and expert hypotheses sufficiently support the proposed vulnerability mechanism and whether the Critic provides sufficient counter-evidence that the mechanism has been neutralized.

Based on this comparison, the Judge produces the **Final Result**, determining whether the target function is vulnerable. CRAFT therefore reaches its final decision through a progressive process of structured causal retrieval, causality-specific hypothesis generation, hypothesis synthesis, and verification rather than through a single direct LLM prediction.

III. EXPERIMENTAL SETUP

This section describes the experimental configuration used to evaluate CRAFT, including the implementation environment, datasets, evaluation metrics, and baseline methods.

A. Implementation Details

All implementations were conducted in Python 3.12. We used LangChain [30] to construct the causality-aware multi-agent reasoning pipeline. To ensure fair comparisons across all experimental settings, including baseline methods and ablation variants, all LLM-based experiments were performed using the same backbone model, gpt-4o-mini [31], with temperature=0.0. For embedding based retrieval and Causality-Typed Vulnerability Knowledge Graph (CTVKG) construction, we employed all-MiniLM-L6-v2, a Sentence-BERT based embedding model [32]. The same embedding model was consistently used for Neighborhood Edge construction, causality-aware evidence retrieval, and retrieval based baseline comparisons to maintain experimental consistency. Unless otherwise specified, the same model configuration and inference settings were applied across CRAFT and all LLM-based baselines.

B. Datasets

We evaluate CRAFT on the PrimeVul benchmark [7], a vulnerability dataset specifically designed for pairwise evaluation between vulnerable functions and their corresponding patched counterparts. Following the original PrimeVul train/validation/test partition, we construct a filtered subset containing eleven CWE categories that collectively span the four causality types considered in this work. The resulting subset consists of 2,786 labeled code samples corresponding to 1,393 vulnerable-patched pairs. Following the original split structure, the filtered dataset contains 2,100 training samples, 352 validation samples, and 334 test samples. Filtering is performed independently within each original split so that no instance is moved across the training, validation, or test partitions.

CWE Category Selection. CWE Category Selection.

The evaluated CWE categories are derived from the failure analysis of our preliminary work, CLEAR [28]. Specifically, we focus on categories for which CLEAR exhibited relatively low pairwise discrimination performance under its unified causal representation. The selection was determined solely from the prior results of CLEAR, before evaluating CRAFT, and was therefore independent of CRAFT's performance. Among these categories, we retain CWEs with sufficient vulnerable-patched pairs for meaningful evaluation and organize them according to the causality types defined in Table I:

- **State**: CWE-476, CWE-415, CWE-401, CWE-835

- **Execution:** CWE-703, CWE-617, CWE-362
- **Constraint:** CWE-189, CWE-369
- **Policy:** CWE-200, CWE-284

The resulting subset therefore provides a challenge oriented evaluation of whether causality-aware modeling can improve upon CLEAR’s unified causal reasoning. We follow the official PrimeVul pairwise evaluation protocol and re-evaluate all baseline methods on exactly the same filtered subset.

C. Evaluation Metrics

Following the pairwise evaluation protocol of PrimeVul [7], we evaluate CRAFT based on its ability to distinguish vulnerable functions from their corresponding patched counterparts.

Detection Metrics. We use pairwise metrics to evaluate whether a model can distinguish a vulnerable function from its corresponding patched version. Pairwise Correct Prediction (P-C) measures the proportion of pairs for which both the vulnerable and patched samples are classified correctly. Pairwise Vulnerable Prediction (P-V) captures cases in which both samples are predicted as vulnerable, whereas Pairwise Benign Prediction (P-B) captures cases in which both are predicted as benign. Pairwise Reversed Prediction (P-R) represents a severe pairwise error, in which the vulnerable sample is predicted as benign while its patched counterpart is predicted as vulnerable. We additionally report the Vulnerability Pair Score ($VPS = P-C - P-R$), which rewards correct pairwise discrimination while explicitly penalizing reversed predictions. P-V and P-B are reported separately to characterize the model’s prediction bias toward vulnerable or benign classifications, respectively. Higher values are preferred for P-C and VPS, whereas lower values are preferred for P-V, P-B, and P-R. All pairwise metrics are reported as percentages (%).

D. Baseline Methods

We compare CRAFT against four groups of baselines: deep learning methods, single-agent prompting, multi-agent reasoning frameworks, and retrieval based vulnerability reasoning approaches. For all LLM-based baselines, we preserve the original reasoning and agent-interaction procedures as closely as possible while using the same backbone LLM (gpt-4o-mini) and evaluation dataset to provide a controlled comparison. All methods are evaluated on the same filtered PrimeVul test split.

Deep Learning Baselines. We evaluate CodeBERT[33] and UniXCoder[34] as representative learning based vulnerability detection models. Both models are fine-tuned on the filtered PrimeVul training split and evaluated using the same pairwise protocol.

Single-Agent Baseline. For single-agent reasoning, we adopt the Chain-of-Thought (CoT) prompting strategy used in PrimeVul [7]. Given the target source code, the LLM performs step-by-step vulnerability reasoning and produces a vulnerability judgment.

Multi-Agent Baselines. We compare against GPTLens[23], VulTrial[24], and CLEAR[28]. GPTLens and VulTrial represent general multi-agent reasoning approaches, while CLEAR serves as the closest causal reasoning baseline for evaluating the benefit of CRAFT’s causality-specific representation.

Retrieval Based Baselines. To examine whether CRAFT’s improvements arise from structured causality-aware structured retrieval rather than retrieval augmentation alone, we additionally evaluate three retrieval based strategies: Instance-Level RAG, Chunk-Level RAG, and Vul-RAG [35].

(1) **Instance-Level RAG** follows a coarse-grained retrieval strategy. Given a target function, the system retrieves the most semantically similar historical vulnerability instance from the training corpus and provides the complete retrieved instance as contextual knowledge for vulnerability reasoning.

(2) **Chunk-Level RAG** performs retrieval at a finer granularity. Instead of retrieving an entire vulnerability instance, the historical vulnerability knowledge is divided into individual evidence chunks, and semantically similar chunks are independently retrieved and aggregated as contextual information.

(3) **Vul-RAG** [35] incorporates vulnerability specific knowledge into LLM-based vulnerability detection by extracting generalizable vulnerability information from historical vulnerability instances and retrieving relevant knowledge during inference. We follow its original retrieval and reasoning procedure while using the same evaluation subset and backbone LLM.

For Instance-Level RAG, Chunk-Level RAG, and CRAFT, all-MiniLM-L6-v2 is used as the embedding model to control for differences in semantic retrieval representations. All retrieval knowledge used by these methods is constructed exclusively from the training split, ensuring that no validation or test information is exposed during retrieval.

IV. EVALUATION

Research Questions. We evaluate CRAFT by answering the following research questions:

- **RQ1:** Does CRAFT improve vulnerability detection over state-of-the-art (SOTA) approaches?
- **RQ2:** How does CRAFT compare with existing retrieval-based approaches?
- **RQ3:** How do individual causal evidence roles contribute to CRAFT’s performance?
- **RQ4:** How do causality-specific representation and expert guided reasoning contribute to CRAFT’s performance?
- **RQ5:** How robust is CRAFT across different LLM backbones?

RQ1: Does CRAFT improve vulnerability detection performance compared with existing SOTA approaches? Table III compares CRAFT with deep-learning, single-agent, and multi-agent vulnerability detection approaches. CRAFT achieves the highest P-C of 22.75 and the highest VPS of 10.17. Compared with CLEAR, our previous causal reasoning framework and the strongest baseline in terms of P-C, CRAFT improves P-C from 14.97 to 22.75, corresponding to a relative improvement of approximately 52.0%. VPS also increases from 5.98 to 10.17, representing an improvement of approximately 70.1%. The improvement in P-C indicates that CRAFT more frequently identifies both the vulnerable and patched versions of a pair correctly, rather than obtaining performance gains by simply favoring one label. This is particularly important for pairwise vulnerability detection because a reliable model should

TABLE III
PAIRWISE VULNERABILITY DETECTION RESULTS ON PRIMEVUL.

Agents	Method	P-C ($\uparrow$)	P-V ($\downarrow$)	P-B ($\downarrow$)	P-R ($\downarrow$)	VPS ($\uparrow$)	Dataset
DL	CodeBERT	3.59	36.52	54.49	5.38	-1.80	PrimeVul
DL	UniXCoder	2.99	27.54	68.26	1.19	1.79	PrimeVul
Single-Agent	CoT	5.98	61.67	23.95	8.38	-2.40	PrimeVul
Multi-Agent	GPTLens	0.59	94.01	2.39	2.99	-2.40	PrimeVul
Multi-Agent	VulTrial	7.18	76.64	8.98	7.18	0.00	PrimeVul
Multi-Agent	CLEAR	14.97	5.98	70.05	8.98	5.98	PrimeVul
Multi-Agent	Ours	22.75	10.77	53.89	12.57	10.17	PrimeVul

TABLE IV
COMPARISON OF RETRIEVAL STRATEGIES.

Method	P-C ($\uparrow$)	P-V ($\downarrow$)	P-B ($\downarrow$)	P-R ($\downarrow$)	VPS ($\uparrow$)	Dataset
Instance-Level RAG	3.59	53.29	25.74	17.36	-13.78	PrimeVul
Chunk-Level RAG	5.38	58.08	17.96	18.56	-13.18	PrimeVul
Vul-RAG	12.57	64.67	13.17	9.58	2.99	PrimeVul
Ours	22.75	10.77	53.89	12.57	10.17	PrimeVul

recognize not only the presence of a vulnerability but also the semantic effect of its corresponding patch. CRAFT nevertheless exhibits a different error distribution from CLEAR. Its P-B decreases from 70.05 to 53.89, indicating that the model is substantially less likely to classify both members of a vulnerable-patched pair as benign. This reduction is accompanied by increases in P-V (5.98 to 10.77) and P-R (8.98 to 12.57), suggesting that the improved pairwise correctness comes with a greater tendency to make vulnerability oriented or reversed predictions in ambiguous cases. Despite these shifts in error distribution, the simultaneous improvements in P-C and VPS indicate that CRAFT achieves substantially stronger pairwise discrimination overall, although reducing reversed predictions remains an area for further improvement.

Answer to RQ1. CRAFT achieves the highest pairwise correctness (P-C = 22.75) and VPS (10.17), improving P-C by approximately 52.0% and VPS by 70.1% over CLEAR. These results show that causality-aware reasoning improves the model’s ability to distinguish vulnerable code from its patched counterpart.

RQ2: How does CRAFT compare with conventional retrieval-based approaches? Table IV compares CRAFT with three retrieval strategies: Instance-Level RAG, Chunk-Level RAG, and Vul-RAG. Instance-Level RAG retrieves complete vulnerability instances based on similarity, while Chunk-Level RAG retrieves independently segmented evidence without explicitly preserving causal relations. Vul-RAG incorporates vulnerability-specific knowledge into retrieval, whereas CRAFT additionally organizes retrieved evidence according to common causal roles with causality-specific semantics and couples it with causality-specific reasoning.

Instance-Level and Chunk-Level RAG achieve only 3.59 and 5.38 P-C, respectively, and both produce negative VPS values (-13.78 and -13.18). These results indicate that simply providing semantically related examples or code fragments does not guarantee reliable pairwise discrimination. Retrieved context can introduce irrelevant or conflicting signals when

the evidence is not aligned with the underlying vulnerability mechanism. Vul-RAG substantially improves P-C to 12.57 and obtains a positive VPS of 2.99, demonstrating the benefit of vulnerability-oriented retrieval. CRAFT further improves P-C to 22.75, corresponding to an 81.0% relative improvement over Vul-RAG, while increasing VPS from 2.99 to 10.17. CRAFT also reduces P-V dramatically from 64.67 to 10.77. Although its P-B is higher than that of Vul-RAG, CRAFT achieves substantially higher P-C and VPS, indicating improved pairwise discrimination despite the increase in P-B. The key distinction is that CRAFT does not treat all retrieved vulnerability evidence as structurally equivalent. Instead, retrieved evidence is organized according to common causal roles while preserving causality-specific semantics. Thus, evidence describing state evolution, execution behavior, constraint satisfaction, or policy enforcement is interpreted within its corresponding causal structure rather than being aggregated solely according to semantic similarity.

Answer to RQ2. CRAFT achieves 81.0% higher P-C than Vul-RAG and substantially outperforms instance and chunk-level retrieval. The results demonstrate that retrieving relevant evidence alone is insufficient and that preserving causality-specific semantics in retrieved evidence substantially improves pairwise reasoning.

RQ3: How do individual causal evidence roles contribute to CRAFT’s performance? Table V evaluates the contribution of the five causal evidence roles by removing each role individually. Removing all retrieved evidence (*No_Evidence*) causes the largest degradation, reducing P-C from 22.75 to 5.98 and VPS from 10.17 to -2.40. This result confirms that CRAFT’s gains cannot be attributed solely to the underlying LLM or multi-agent prompting. Among individual roles, removing *Mechanism* and *Violation* produces substantial degradation, reducing P-C to 16.16 and 15.56, respectively. In particular, removing *Mechanism* results in a negative VPS (-1.20), highlighting the importance of modeling the operation through which unsafe behavior emerges. Removing *Entity*,

TABLE V
ABLATION STUDY

Method	P-C ($\uparrow$)	P-V ($\downarrow$)	P-B ($\downarrow$)	P-R ($\downarrow$)	VPS ($\uparrow$)	Dataset
No_Evidence	5.98	22.75	62.87	8.38	-2.40	PrimeVul
No_Entity	17.96	19.16	49.10	13.77	4.19	PrimeVul
No_Context	18.56	9.58	57.48	14.37	4.19	PrimeVul
No_Mechanism	16.16	8.98	57.48	17.36	-1.20	PrimeVul
No_Violation	15.56	13.17	58.08	13.17	2.39	PrimeVul
No_Fix_Intent	16.76	5.98	63.47	13.77	2.99	PrimeVul
Full	22.75	10.77	53.89	12.57	10.17	PrimeVul

TABLE VI
ABLATION STUDY OF CRAFT’S CAUSALITY-SPECIFIC ARCHITECTURE ON PRIMEVUL.

Method	P-C ($\uparrow$)	P-V ($\downarrow$)	P-B ($\downarrow$)	P-R ($\downarrow$)	VPS ($\uparrow$)
Unified_Causal_Reasoning	4.19	0.59	90.41	4.79	-0.60
Without_Experts	13.77	4.19	71.25	10.77	2.99
Full	22.75	10.77	53.89	12.57	10.17

Context, or *Fix_Intent* also degrades performance, indicating that the five causal roles provide complementary evidence rather than interchangeable information.

Answer to RQ3. All five causal evidence roles contribute to CRAFT’s performance, with Mechanism and Violation having particularly strong effects on pairwise discrimination. Removing causal evidence entirely reduces P-C by 73.7%, confirming the importance of structured causal grounding.

RQ4: How do causality-specific representation and expert guided reasoning contribute to CRAFT’s performance? Table VI isolates the two architectural components that distinguish CRAFT’s causality-aware reasoning: causality-specific representation and expert guided reasoning. *Unified Causal Reasoning* replaces the causality-specific representation with a common causal abstraction across all causality types, while *Without Experts* retains the causality-specific representation but removes the causality-specific expert reasoning stage. Replacing the causality-specific representation with a unified causal representation reduces P-C from 22.75 to 4.19 and VPS from 10.17 to -0.60. This substantial degradation indicates that representing heterogeneous vulnerability mechanisms through distinct causal structures is critical for reliable pairwise discrimination. Retaining the causality-specific representation but removing the Security Experts Group also reduces P-C from 22.75 to 13.77 and VPS from 10.17 to 2.99. Thus, causality-specific representation is most effective when it actively guides the reasoning process rather than serving only as a representation or retrieval label.

Answer to RQ4. Both causality-specific representation and expert guided reasoning are critical to CRAFT. A unified causal representation reduces P-C from 22.75 to 4.19, while removing the expert reasoning stage reduces it to 13.77, demonstrating that CRAFT benefits from both causality-specific representation and expert guided reasoning.

RQ5: How robust is CRAFT across different LLM backbones?

To examine whether CRAFT depends on a particular LLM, we evaluate the full framework using four backbone models: *gemini-3.1-flash-lite* [36], *gemini-2.5-flash* [37], *gpt-4o-mini* [31], and *gpt-4.1-nano* [38], while keeping all other experimental settings unchanged.

As shown in Table VII, CRAFT maintains P-C values between 18.56 and 24.55 across the four backbones. The results also reveal different pairwise error profiles across backbones. *gpt-4o-mini* achieves the lowest P-V (10.77) but exhibits a relatively high P-B (53.89), whereas *gemini-2.5-flash* shows the opposite tendency, achieving the lowest P-B (10.77) but the highest P-V (52.69). This contrast indicates that the underlying LLM affects the distribution of pairwise errors even when the CTVKG, retrieval procedure, and multi-agent reasoning architecture remain unchanged. Nevertheless, CRAFT maintains pairwise vulnerability discrimination across all evaluated backbones, indicating that its effectiveness is not specific to the default *gpt-4o-mini* backbone.

Answer to RQ5. CRAFT remains effective across all four evaluated LLM backbones, with P-C ranging from 18.56 to 24.55. Although backbone choice affects the distribution of pairwise errors, CRAFT’s effectiveness is not limited to the default *gpt-4o-mini* backbone.

A. Limitations and Future Directions

Data contamination risk. Since PrimeVul’s CVEs were publicly disclosed before our experiments, they may overlap with *gpt-4o-mini*’s pretraining corpus, raising the possibility that predictions partly reflect memorization rather than genuine reasoning. This risk is partially mitigated by our pairwise protocol, which requires distinguishing vulnerable from near-identical patched code, and by the fact that all LLM-based baselines share the same backbone, preserving the validity of relative comparisons. A temporally-controlled evaluation using post-cutoff CVEs or open-weight models

TABLE VII
PAIRWISE VULNERABILITY DETECTION PERFORMANCE OF CRAFT WITH DIFFERENT LLM BACKBONES ON PRIMEVUL.

Model	P-C ($\uparrow$)	P-V ($\downarrow$)	P-B ($\downarrow$)	P-R ($\downarrow$)	VPS ($\uparrow$)
gemini-3.1-flash-lite	18.56	42.51	24.55	14.37	4.19
gemini-2.5-flash	21.55	52.69	10.77	14.97	6.58
gpt-4o-mini	22.75	10.77	53.89	12.57	10.17
gpt-4.1-nano	24.55	32.93	22.15	20.35	4.19

would more rigorously address this concern, which we leave to future work.

Inter-procedural scope. CRAFT constructs CTVKG from function-level vulnerability instances and reasons over one function at a time; it therefore does not explicitly model cross-file data flows, inter-procedural call chains, or framework-level behaviors. This limitation particularly affects vulnerabilities whose evidence spans multiple procedures. Extending CTVKG to a repository scale representation incorporating call graphs, data-flow graphs, and cross-file dependency analysis is a natural direction for capturing vulnerability mechanisms that span larger code contexts.

Semantic similarity bias in graph construction. Neighborhood Edges are constructed using cosine similarity over semantic embeddings. While computationally efficient, semantic similarity does not always guarantee causal consistency: nodes with similar textual descriptions may be connected even when causally unrelated, while nodes describing the same underlying mechanism may remain disconnected if their textual descriptions differ. We mitigate this with an LLM-based validation step during Global Edge integration, but the underlying limitation of similarity-based preprocessing remains. Future work could explore hybrid similarity metrics that incorporate structural signals such as AST patterns, data-flow dependencies, or API usage patterns or apply graph contrastive learning to learn causally consistent neighbor relationships directly from vulnerability data.

Taxonomy coverage. The current causality taxonomy targets representative CWE categories mapped to four dominant verification requirements. Emerging or compound vulnerability types may not fit cleanly into a single dominant mechanism, which motivates future work on continual knowledge graph updates and adaptive category discovery to support previously unseen vulnerability patterns.

Toward automated vulnerability repair. This work focuses on vulnerability detection, but the causal structure captured by CTVKG naturally encodes mitigation information: the relationship between *mechanism/violation* nodes and their corresponding *fix_intent* node describes how a given vulnerability class is resolved. Extending this relationship could support automated repair for instance, learning repair patterns conditioned on mechanism type. We leave the design of a knowledge-graph-based repair agent that integrates repair generation into an end-to-end pipeline for future work.

V. RELATED WORK

Single-Agent Vulnerability Detection. Recent studies have actively explored deep learning (DL) and large language

model (LLM) based approaches for automated vulnerability detection. Early DL-based models focus on learning semantic representations of source code to identify vulnerable patterns. For example, VulBERTa[5] leverages vulnerability-aware tokenization and pre-training to capture vulnerability-related contexts in code, thereby improving detection accuracy and generalization capability. LineVul[17] introduces a Transformer-based architecture for line-level vulnerability prediction, enabling fine-grained localization of vulnerable statements. Additionally, Steenhoek et al[6]. propose DeepDFA, which incorporates data-flow analysis into graph-based deep learning models to improve vulnerability detection effectiveness. With the emergence of large language models, several studies have investigated their applicability to vulnerability detection. Tamberg and Bahsi [15] conduct a comprehensive benchmarking study on LLM-based vulnerability detection and show that carefully designed prompting strategies can significantly influence model performance. Similarly, Zhang et al[14]. demonstrate that prompt engineering techniques can effectively enhance vulnerability detection performance when applied to LLMs such as ChatGPT. Zhou et al[16]. further explore the use of in-context learning with LLMs for vulnerability detection and report that GPT-based models can achieve competitive performance when provided with appropriate contextual knowledge. Despite these advances, recent studies highlight fundamental limitations of LLM-based vulnerability detection. Ding et al [7]. show that code language models struggle to distinguish vulnerable code from its patched counterpart when the differences between them are subtle. This observation suggests that vulnerability detection cannot be reliably addressed solely through token-level similarity or statistical correlations. To alleviate this limitation, several studies incorporate external vulnerability knowledge into the detection process [39].

Retrieval-Augmented Generation for Vulnerability Detection. To overcome the limitations of standalone LLMs, Retrieval-Augmented Generation (RAG)[18] has been proposed to incorporate external knowledge into the reasoning process. Liu et al[39]. propose VUL-GPT, which combines TF-IDF/BM25-based code retrieval with GPT-based analysis using in-context learning, significantly improving recall and F1 scores on vulnerability detection benchmarks. Similarly, Sheng et al[40]. integrate RAG with chain-of-thought reasoning using GPT-4o, demonstrating improved vulnerability detection performance. More recently, Du et al[35]. introduced Vul-RAG, which constructs a structured vulnerability knowledge base consisting of functional semantics, vulnerability causes, and fixing solutions, enabling models to distinguish vulnerable code from patched code pairs better.

Multi-Agent Vulnerability Detection. More recently, multi-agent architectures have been proposed to address reasoning limitations in single-model LLM systems. These approaches assign complementary roles to multiple agents to encourage diverse reasoning perspectives and reduce overconfidence in individual model outputs [41]. For example, EvalSVA[42] organizes agents as security experts who collaboratively analyze potential vulnerabilities. MuCoLD[43] assigns tester and developer roles to simulate collaborative vulnerability analysis. Similarly, LLM-SmartAudit [44] employs role-playing and collaborative reasoning to detect vulnerabilities in smart contracts. iAudit [45] further extends this paradigm by introducing specialized agents such as a detector, reasoner, and critic to produce interpretable vulnerability assessments. GPTLens[23] deploys multiple inspection agents together with a critic agent to verify vulnerability findings. VulTrial[24] proposes a courtroom-inspired framework where agents assume roles such as prosecutor, defense attorney, and judge to debate the presence of vulnerabilities.

Knowledge Graphs in Cybersecurity. Meanwhile, to enable structured representation and integration of cybersecurity knowledge, several studies have explored knowledge graph-based approaches in vulnerability and threat intelligence analysis. Knowledge graphs enable the integration of heterogeneous security information sources and support structured reasoning over relationships among vulnerability entities. For example, CTI-KG [46] constructs a Cyber Threat Intelligence knowledge graph from threat reports to represent relationships among threat actors, targets, and attack tools. In the vulnerability domain, VulKG [47] builds a vulnerability-oriented knowledge graph from National Vulnerability Database (NVD) descriptions using ontology-guided entity extraction and reasoning to discover hidden weakness relations across CWE categories. Similarly, Høst et al.[48] construct a knowledge graph by extracting entities and relations directly from textual vulnerability descriptions to support structured representation and querying of vulnerability information. Although these approaches demonstrate the effectiveness of knowledge graphs for organizing vulnerability-related information, they primarily focus on semantic relationships between vulnerability entities rather than explicitly modeling the causal structure underlying vulnerability formation and propagation. Our preliminary work, CLEAR [28] addresses this limitation by introducing a Vulnerability Causal Knowledge Graph that grounds multi-agent reasoning in a unified causal representation. While CLEAR demonstrates the effectiveness of causal evidence for vulnerability detection, it applies the same causal abstraction across vulnerability categories. Building on this foundation, CRAFT introduces causality-specific representations for state, execution, constraint, and policy failures, coupled with category-aware retrieval and specialized expert reasoning.

VI. CONCLUSION

This paper presented CRAFT, a Category-aware Reasoning and Adaptive Framework for Vulnerability Types that combines a Causality-Typed Vulnerability Knowledge Graph

(CTVKG) with LLM-based multi-agent reasoning for software vulnerability detection. Unlike approaches that rely primarily on implicit semantic reasoning or unified causal representations, CRAFT explicitly distinguishes vulnerability mechanisms according to four causality types: *State*, *Execution*, *Constraint*, and *Policy*, and adapts evidence representation and reasoning accordingly.

CRAFT introduces Causality-Typed Root Cause Grounding to represent vulnerabilities through common causal roles *Entity*, *Context*, *Mechanism*, *Violation*, and *Fix Intent* while preserving causality-specific semantics within the CTVKG. During inference, the framework retrieves structured causal evidence and employs causality-specific experts together with Claim, Critic, and Judge agents to generate vulnerability hypotheses, synthesize candidate claims, evaluate counter-evidence, and produce a final evidence grounded decision.

Experiments on the PrimeVul benchmark demonstrate that CRAFT improves pairwise vulnerability discrimination over the evaluated baselines, achieving a P-C of 22.75% and a VPS of 10.17. The ablation studies further show that both causality-specific representation and causality-specific expert reasoning contribute substantially to the performance of CRAFT, while the five causal evidence roles provide complementary information for vulnerability reasoning. In addition, CRAFT maintains consistent pairwise discrimination across multiple LLM backbones, indicating that its effectiveness is not specific to a single backbone model.

Overall, these results demonstrate that explicitly preserving causality-specific semantics in both vulnerability knowledge representation and multi-agent reasoning provides an effective direction for improving fine-grained discrimination between vulnerable functions and their patched counterparts.

A. Data Availability

The filtered PrimeVul subset, the constructed Causality-Typed Vulnerability Knowledge Graph (CTVKG), and the source code for CRAFT, including the Causality-Typed Root Cause Grounding and multi-agent reasoning pipeline, will be made publicly available upon publication.

REFERENCES

- [1] W. Charoenwet, P. Thongtanunam, V.-T. Pham, and C. Treude, "Toward effective secure code reviews: An empirical study of security-related coding weaknesses," *Empirical Software Engineering*, vol. 29, no. 4, pp. 1–47, 2024.
- [2] G. Díaz and J. R. Bermejo, "Static analysis of source code security: Assessment of tools against samate tests," *Information and software technology*, vol. 55, no. 8, pp. 1462–1476, 2013.
- [3] A. Kaur and R. Nayyar, "A comparative study of static code analysis tools for vulnerability detection in c/c++ and java source code," *Procedia Computer Science*, vol. 171, pp. 2023–2029, 2020, third International Conference on Computing and Network Communications (CoCoNet'19). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050920312023>
- [4] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 05, pp. 2224–2236, Sep. 2021. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TDSC.2019.2942930>
- [5] H. Hanif and S. Maffei, "Vulberta: Simplified source code pre-training for vulnerability detection," in *2022 International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8.

- [6] B. Steenhoeck, H. Gao, and W. Le, "Dataflow analysis-inspired deep learning for efficient vulnerability detection," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3597503.3623345>
- [7] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ser. ICSE '25. IEEE Press, 2025, p. 1729–1741. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00038>
- [8] M. M. Rahman, I. Ceka, C. Mao, S. Chakraborty, B. Ray, and W. Le, "Towards causal deep learning for vulnerability detection," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3597503.3639170>
- [9] N. Risse and M. Böhme, "Uncovering the limits of machine learning for automatic vulnerability detection," in *Proceedings of the 33rd USENIX Conference on Security Symposium*, ser. SEC '24. USA: USENIX Association, 2024.
- [10] N. Huynh and B. Lin, "Large language models for code generation: A comprehensive survey of challenges, techniques, evaluation, and applications," *arXiv preprint arXiv:2503.01245*, 2025.
- [11] Z. Zheng, K. Ning, Y. Wang, J. Zhang, D. Zheng, M. Ye, and J. Chen, "A survey of large language models for code: Evolution, benchmarking, and future trends," *arXiv preprint arXiv:2311.10372*, 2023.
- [12] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLms in software security: A survey of vulnerability detection techniques and insights," *ACM Computing Surveys*, 2025, arXiv:2502.07049.
- [13] S. Kaniewski, F. Schmidt, M. Enzweiler, M. Menth, and T. Heer, "A systematic literature review on detecting software vulnerabilities with large language models," *ACM Trans. Softw. Eng. Methodol.*, May 2026, just Accepted. [Online]. Available: <https://doi.org/10.1145/3815425>
- [14] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, and H. Li, "Prompt-enhanced software vulnerability detection using chatgpt," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, ser. ICSE-Companion '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 276–277. [Online]. Available: <https://doi.org/10.1145/3639478.3643065>
- [15] K. Tamberg and H. Bahsi, "Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study," *IEEE Access*, vol. 13, pp. 29 698–29 717, 2025.
- [16] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. [Online]. Available: <https://doi.org/10.1145/3708522>
- [17] M. Fu and C. Tantithamthavorn, "Linevul: a transformer-based line-level vulnerability prediction," in *Proceedings of the 19th International Conference on Mining Software Repositories*, ser. MSR '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 608–620. [Online]. Available: <https://doi.org/10.1145/3524842.3528452>
- [18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [19] M. Cheng, Y. Luo, J. Ouyang, Q. Liu, H. Liu, L. Li, S. Yu, B. Zhang, J. Cao, J. Ma *et al.*, "A survey on knowledge-oriented retrieval-augmented generation," *arXiv preprint arXiv:2503.10677*, 2025.
- [20] N. Wang, X. Han, J. Singh, J. Ma, and V. Chaudhary, "CausalRAG: Integrating causal graphs into retrieval-augmented generation," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 22 680–22 693. [Online]. Available: <https://aclanthology.org/2025.findings-acl.1165/>
- [21] X. Zhu, Y. Xie, Y. Liu, Y. Li, and W. Hu, "Knowledge graph-guided retrieval augmented generation," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, L. Chiruzzo, A. Ritter, and L. Wang, Eds. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 8912–8924. [Online]. Available: <https://aclanthology.org/2025.naacl-long.449/>
- [22] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, vol. 43, no. 2, Jan. 2025. [Online]. Available: <https://doi.org/10.1145/3703155>
- [23] S. Hu, T. Huang, F. İlhan, S. F. Tekin, and L. Liu, "Large language model-powered smart contract vulnerability detection: New perspectives," in *2023 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*. IEEE, 2023, pp. 297–306.
- [24] R. Widayarsi, M. Weyssow, I. C. Irsan, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang, and D. Lo, "Let the trial begin: A mock-court approach to vulnerability detection using llm-based agents," *arXiv preprint arXiv:2505.10961*, 2025.
- [25] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML '24. JMLR.org, 2024.
- [26] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, "The rise and potential of large language model based agents: A survey," *Science China information sciences*, vol. 68, no. 2, p. 121101, 2025.
- [27] Y. Cui, H. Fu, H. Zhang, L. Wang, and C. Zuo, "Free-MAD: Consensus-free multi-agent debate," in *Findings of the Association for Computational Linguistics: ACL 2026*, M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, Eds. San Diego, California, United States: Association for Computational Linguistics, Jul. 2026, pp. 31 977–31 997. [Online]. Available: <https://aclanthology.org/2026.findings-acl.1600/>
- [28] S. Yun, S. Hwang, Y. Lee, K. Kang, and S. Park, "Clear: Causal context-based agentic reasoning for vulnerability detection," *arXiv preprint arXiv:2608.03134*, 2026.
- [29] MITRE Corporation, "Common weakness enumeration (cwe)," <https://cwe.mitre.org>, accessed 2026.
- [30] H. Chaitin, "Langchain," 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [31] OpenAI, "Gpt-4o mini: Advancing cost-efficient intelligence," <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, Jul. 2024, accessed: 2026-08-10.
- [32] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992. [Online]. Available: <https://aclanthology.org/D19-1410/>
- [33] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 1536–1547. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.139/>
- [34] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 7212–7225. [Online]. Available: <https://aclanthology.org/2022.acl-long.499/>
- [35] X. Du, G. Zheng, K. Wang, Y. Zou, Y. Wang, W. Deng, J. Feng, M. Liu, B. Chen, X. Peng, T. Ma, and Y. Lou, "Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag," *ACM Trans. Softw. Eng. Methodol.*, Feb. 2026, just Accepted. [Online]. Available: <https://doi.org/10.1145/3797277>
- [36] G. DeepMind, "Gemini 3.1 Flash-Lite: Model Card," 2026, accessed: 2026-08-10. [Online]. Available: <https://deepmind.google/models/model-cards/gemini-3-1-flash-lite/>
- [37] Google. (2025) Gemini 2.5 flash — gemini api. [Online]. Available: <https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash>
- [38] OpenAI, "GPT-4.1 nano (large language model)," 2025, accessed: 2026-08-10. [Online]. Available: <https://openai.com/index/gpt-4-1/>
- [39] Z. Liu, Q. Liao, W. Gu, and C. Gao, "Software vulnerability detection with gpt and in-context learning," in *2023 8th International Conference on Data Science in Cyberspace (DSC)*, 2023, pp. 229–236.
- [40] Z. Sheng, F. Wu, X. Zuo, C. Li, Y. Qiao, and H. Lei, "Research on the llm-driven vulnerability detection system using lprotector," in *2024 IEEE*

4th International Conference on Data Science and Computer Application (ICDSCA), 2024, pp. 192–196.

- [41] J. He, C. Treude, and D. Lo, “Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. [Online]. Available: <https://doi.org/10.1145/3712003>
- [42] X.-C. Wen, J. Ye, C. Gao, L. Wu, and Q. Liao, “Evalsva: Multi-agent evaluators for next-gen software vulnerability assessment,” *arXiv preprint arXiv:2501.14737*, 2024.
- [43] Z. Mao, J. Li, D. Jin, M. Li, and K. Tei, “Multi-role consensus through llms discussions for vulnerability detection,” in *2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*, 2024, pp. 1318–1319.
- [44] Z. Wei, J. Sun, Y. Sun, Y. Liu, D. Wu, Z. Zhang, X. Zhang, M. Li, Y. Liu, C. Li, M. Wan, J. Dong, and L. Zhu, “Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems,” *IEEE Transactions on Software Engineering*, vol. 51, no. 10, pp. 2830–2846, Oct. 2025. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TSE.2025.3597319>
- [45] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, “Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications,” in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ser. ICSE ’25. IEEE Press, 2025, p. 1742–1754. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00027>
- [46] N. Rastogi, S. Dutta, A. Gittens, M. J. Zaki, and C. Aggarwal, “Tinker: A framework for open source cyberthreat intelligence,” in *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2022, pp. 1569–1574.
- [47] J. Yin, W. Hong, H. Wang, J. Cao, Y. Miao, and Y. Zhang, “A compact vulnerability knowledge graph for risk assessment,” *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 8, Jul. 2024. [Online]. Available: <https://doi.org/10.1145/3671005>
- [48] A. Høst, P. Lison, and L. Moonen, “Constructing a knowledge graph from textual descriptions of software vulnerabilities in the national vulnerability database,” in *Proceedings of the 24th Nordic Conference on Computational Linguistics (NoDaLiDa)*, T. Alumäe and M. Fishel, Eds. Tórshavn, Faroe Islands: University of Tartu Library, May 2023, pp. 386–391. [Online]. Available: <https://aclanthology.org/2023.nodalida-1.40/>



Sijune Hwang is currently pursuing the M.S. degree with the Department of Information Security, Hanyang University, Seoul, Republic of Korea, where he is a member of the Hanyang AI & Cyber Security Lab. His research interests include software security, large language models (LLMs). His current research focuses on developing trustworthy AI techniques for software vulnerability detection.



Sungju Yun is currently pursuing the Ph.D. degree with the College of Computing at Hanyang University, South Korea, where he is a member of the Hanyang AI & Cyber Security Lab. His research interests encompass cybersecurity, program analysis, automated software engineering, and privacy compliance. His current research focuses on leveraging Large Language Models (LLMs) and agentic frameworks for automated privacy non-compliance verification and software vulnerability detection in complex ecosystems.



Hong-In Won received the Ph.D. degree in mechanical design engineering from Hanyang University, Seoul, Republic of Korea. He also conducted his postdoctoral research in the field of virtual engineering at the Technical University of Dresden (TU Dresden), Germany. He is currently a Principal Researcher with the Manufacturing AI Research Center at the Korea Institute of Industrial Technology (KITECH). His research interests focus on autonomous manufacturing technologies based on AI and digital twins, with a particular emphasis on developing secure and trustworthy manufacturing intelligence. He is actively involved in strategic planning for the AI transformation of the manufacturing industry, focusing on the integration of resilient cyber-physical systems (CPS) and secure AI-Digital Twin platforms for next-generation industrial applications.



Junggab Son received the BSE degree in computer science and engineering from Hanyang University, Ansan, South Korea (2009), and the Ph.D. degree in computer science and engineering from Hanyang University, Seoul, South Korea (2014). From 2014 to 2016, he was a Post-doctoral Research Associate with the Department of Math and Physics, North Carolina Central University. From 2016 to 2018, he was a Research Fellow and a Limited-term Assistant Professor at Kennesaw State University. From 2018 to 2022, he was an Assistant Professor of Computer Science at Kennesaw State University. Currently, he is an Assistant Professor of Computer Science and a Director of Information and Intelligent Security (IIS) Laboratory at University of Nevada, Las Vegas. His research interests include applied cryptography, privacy preservation, blockchain and smart contract, AI-powered cybersecurity solutions, and security/privacy issues in artificial intelligent algorithms.



Yeonjoon Lee received his B.S. degree from Hanyang University, Seoul, Republic of Korea, in 2012, and his Ph.D. degree in security informatics from Indiana University Bloomington, Bloomington, IN, USA, in 2019. He is currently an Assistant Professor with the College of Computing, Hanyang University, Ansan, Republic of Korea. His research interests include mobile security, usable security, cybercrime, and AI security.



Sungbin Park received the B.S. degree from Hanyang University, Ansan, Republic of Korea, in 2022. He is currently pursuing the Ph.D. degree with Hanyang University, Seoul, Republic of Korea. His research interests include AI security, usable security, and differential privacy.